Hands-On Lab: Creating a Custom Event-Driven Timer System in C#

Objective

Learn how to use delegates and events by building a custom timer system. This system will:

1. Allow subscribers to be notified at regular intervals.
2. Support multiple subscribers with different response behaviors.

By the end of this lab, you will understand how to use events and delegates to implement a flexible and reusable timer component.

Scenario

You will build a `CustomTimer` class that uses events to notify subscribers at specified intervals. Different components (e.g., logging, UI updates) can subscribe to these notifications.

Step 1: Define the `CustomTimer` Class

The `CustomTimer` class will encapsulate timer logic and raise events at regular intervals.

Instructions

1. Create a new class `CustomTimer`.
2. Use a `Timer` from the `System.Timers` namespace to trigger interval-based events.
3. Define a delegate `TickHandler` for the timer tick events.
4. Add an event `Tick` using the `TickHandler` delegate.
5. Provide methods to start and stop the timer.

Code

```
using System;
using System.Timers;

public class CustomTimer
{
    private readonly Timer timer;

    // Define a delegate for the Tick event
```

```
    public delegate void TickHandler(DateTime currentTime);

    // Declare the Tick event
    public event TickHandler? Tick;

    public CustomTimer(double interval)
    {
        timer = new Timer(interval);
        timer.Elapsed += OnTimedEvent;
    }

    public void Start()
    {
        timer.Start();
        Console.WriteLine("Timer started.");
    }

    public void Stop()
    {
        timer.Stop();
        Console.WriteLine("Timer stopped.");
    }

    private void OnTimedEvent(object? sender, ElapsedEventArgs e)
    {
        Tick?.Invoke(DateTime.Now);
    }
}
```

Step 2: Create Subscriber Classes

Create subscriber classes that will react to timer events in different ways.

Instructions

1. Create a `Logger` class to log timer ticks.
2. Create a `UIUpdater` class to simulate updating a user interface.

Code

```
public class Logger
{
    public void LogTime(DateTime currentTime)
    {
        Console.WriteLine($"[Logger] Tick at: {currentTime}");
    }
```

```
}

public class UIUpdater
{
    public void UpdateUI(DateTime currentTime)
    {
        Console.WriteLine($"[UIUpdater] UI updated at: {currentTime}");
    }
}
```

Step 3: Test the Timer System

Write a program to test the `CustomTimer` and subscriber classes.

Instructions

1. Create instances of `Logger` and `UIUpdater`.
2. Subscribe their methods to the `CustomTimer.Tick` event.
3. Start the timer and observe the output.
4. Unsubscribe the `Logger` from the event and observe the changes.

Code

```
using System;

class Program
{
    static void Main()
    {
        // Create the timer with a 1-second interval
        var timer = new CustomTimer(1000);

        // Create subscribers
        var logger = new Logger();
        var uiUpdater = new UIUpdater();

        // Subscribe to the timer's Tick event
        timer.Tick += logger.LogTime;
        timer.Tick += uiUpdater.UpdateUI;

        // Start the timer
        timer.Start();

        Console.WriteLine("Press Enter to unsubscribe Logger...");
        Console.ReadLine();
```

```
        // Unsubscribe Logger
        timer.Tick -= logger.LogTime;

        Console.WriteLine("Press Enter to stop the timer...");
        Console.ReadLine();

        // Stop the timer
        timer.Stop();
    }
}
```

Step 4: Add Enhancements (Optional Exercises)

Exercise 1: Add Timer Completion

Modify the `CustomTimer` to raise a `Completed` event when it has executed a specified number of ticks.

Exercise 2: Custom Interval Logic

Allow subscribers to specify their own logic for how often they want to receive notifications (e.g., every 2nd tick).

Exercise 3: Timer Statistics

Track and display statistics about the number of ticks and elapsed time.

Summary

- **Delegates**: Allow you to specify the method signature for event handlers.
- **Events**: Provide a mechanism for loosely coupled communication between objects.
- **Timers**: Can be combined with events to build dynamic and reusable systems.